

EMBEDDED MACHINE LEARNING · PHD COURSE

SESSION 01

Embedded systems and the case for machine learning at the edge

What is scarce on a small device, why it is still worth running learned models there, and the five numbers every later session prices its techniques against.

THE FRAMING

A model that is 94 % accurate and does not fit is 0 % useful.

Feasibility here is a hard constraint set, not a soft preference.
The interesting content of the field is how accuracy trades
against memory, latency and energy along that boundary.

HOW THIS COURSE WORKS

Four lectures, one presentation session, two layers of material

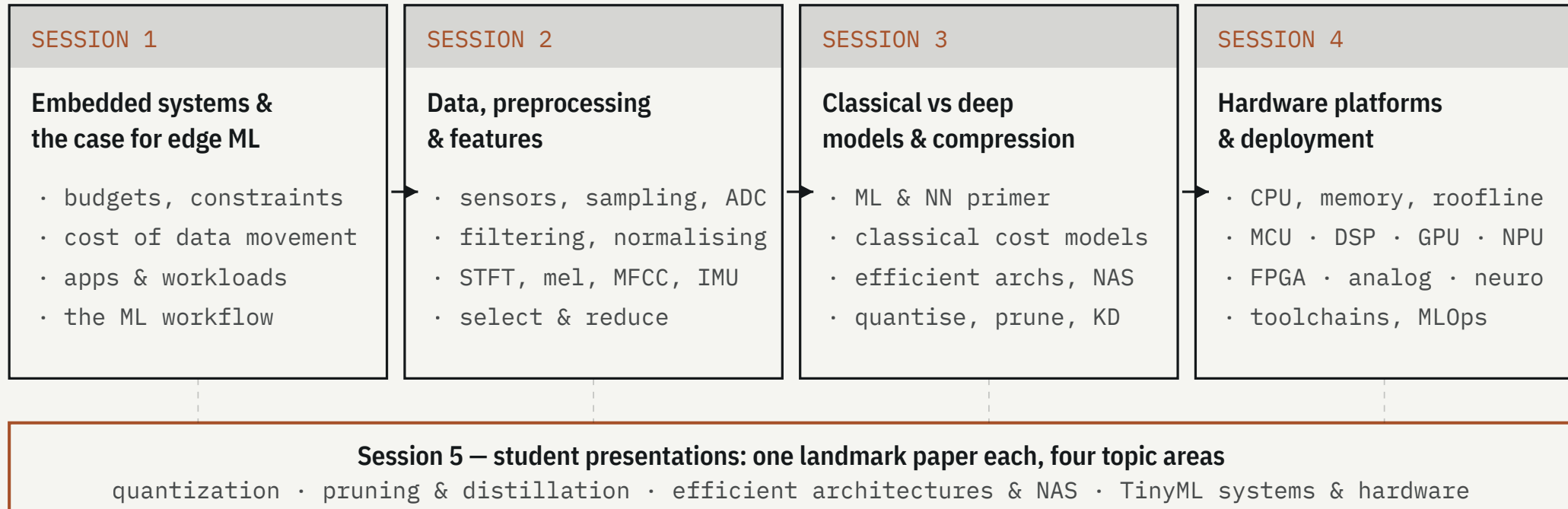
- **Sessions 1–4:** the pipeline from sensor to silicon — the device and its budgets, the data, the models, the hardware.
- **Session 5:** three or four of you present one landmark paper each, from four topic areas.
- **Web chapters** go further than the lecture: **primer** sections for missing background, **depth** sections for the full argument.
- Choose your presentation paper by the **end of Session 2**.

If ML is new to you: read the primer sections **before** each lecture. If hardware is new to you: the same.

THE MAP

The course follows the pipeline

four lectures × 90 min – the order follows the pipeline from sensor to silicon



every session is priced against the same five budgets: flash · SRAM · MACs · latency · energy per inference

METHOD

Four questions we ask of every technique

- **What does it cost?** Count MACs, weight bytes, **peak activation bytes**, and bytes moved.
- **Where does the cost go?** Arithmetic is nearly free; data movement is not.
- **What does it cost in accuracy — and why?** Mechanisms, not folklore.
- **How would you know?** Which device, compiler, clock, how many repetitions, what dispersion?

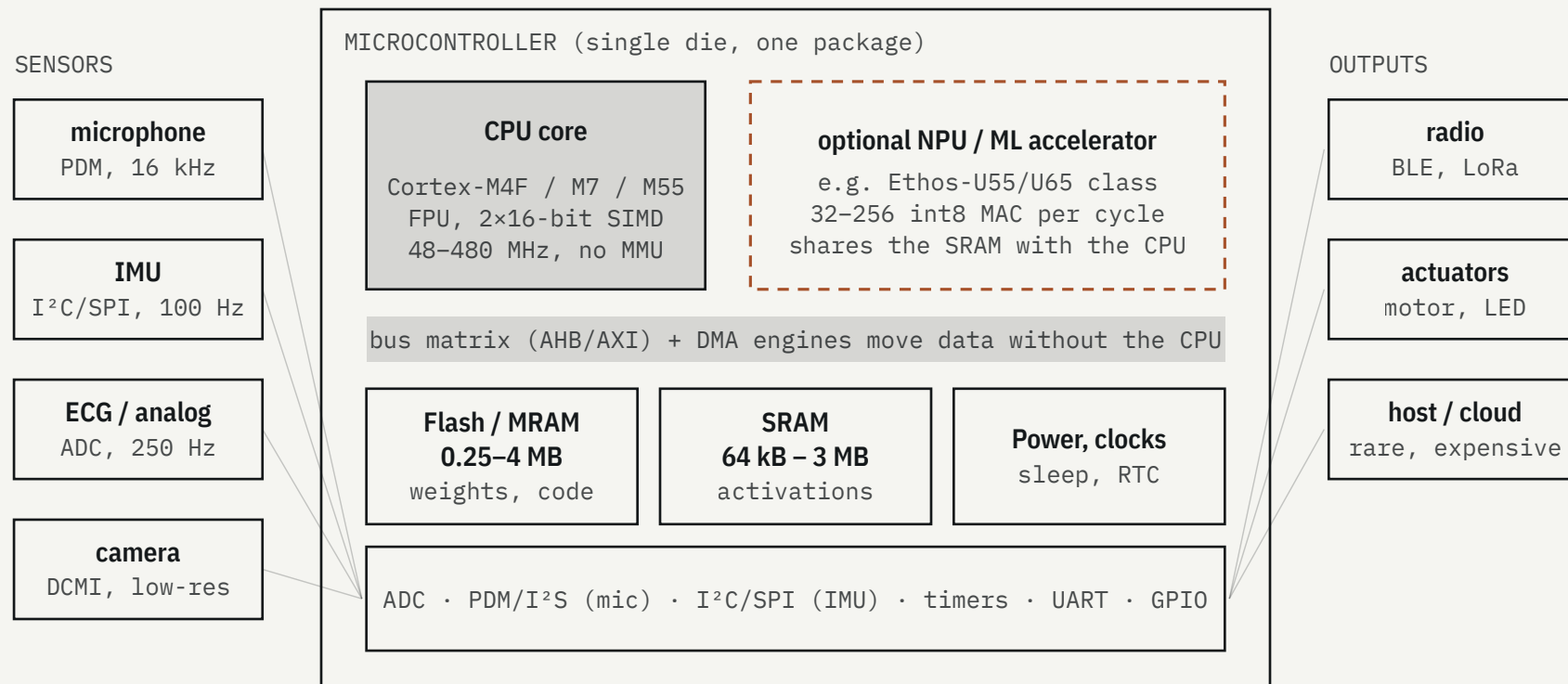
01

Embedded systems, in plain terms

What is actually inside the device — seen by someone who wants to run a model on it.

ANATOMY

A microcontroller, as an ML engineer sees it



Typical figures, not a specific part – always read the datasheet of the device you have.

FIVE FEATURES

What shapes everything that follows

- **Two small memories:** flash (weights, code) and SRAM (activations, buffers) — kB to a few MB.
- **No operating system**, or a tiny real-time one: no virtual memory, often no heap.
- **A modest core:** tens to hundreds of MHz, with SIMD that does two 16-bit MACs per instruction.
- **Peripherals and DMA:** samples arrive while the core sleeps.
- **Power modes:** average power is set by how rarely and how briefly it wakes.

ORDERS OF MAGNITUDE

Numbers to carry around

FLASH 0.25–4 MB weights and program	SRAM 32 kB–3 MB activations, buffers, stack	CLOCK 48–480 MHz Cortex-M class core	POWER μW–100 mW sleep to full speed
---	---	--	--

A laptop has $\sim 10^4\times$ the memory and $\sim 10^3\times$ the power. Nothing transfers down without checking.

01

Why this field exists

A hardware fact from 2004 decides the syllabus of a machine-learning course in 2026.

FOUNDATIONS

Where the constraint comes from

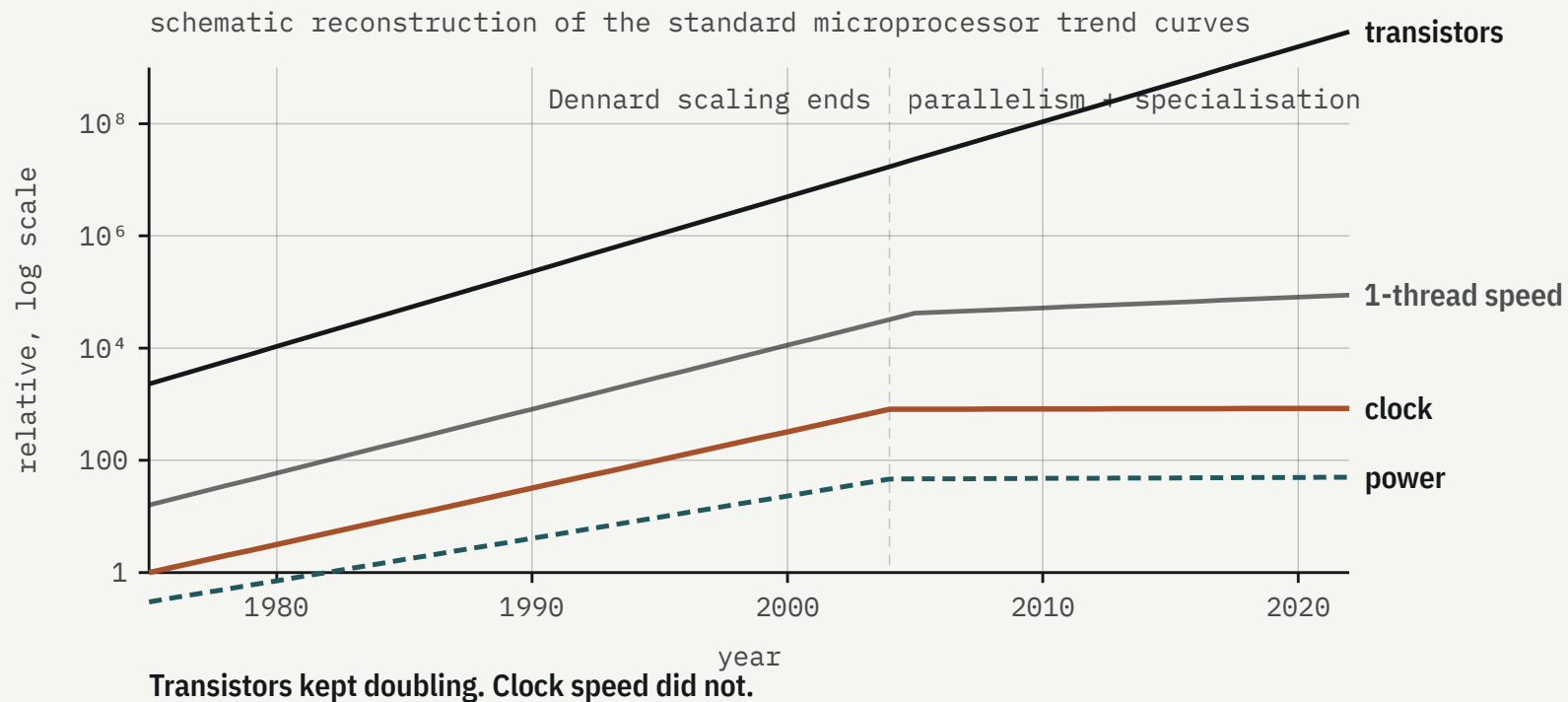
$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f$ dynamic: paid on every switch

$P_{\text{stat}} = \text{leakage}$ paid whether you switch or not

Dennard scaling: shrinking a transistor also let you shrink V and C , so each generation ran faster at the same power. It ended in the mid-2000s, because below roughly 1 V the threshold voltage cannot fall further without leakage exploding.

FOUNDATIONS

Transistors kept doubling. Clock speed did not.



Everything after 2004 is **parallelism** and **specialisation** – because the free speed stopped arriving.

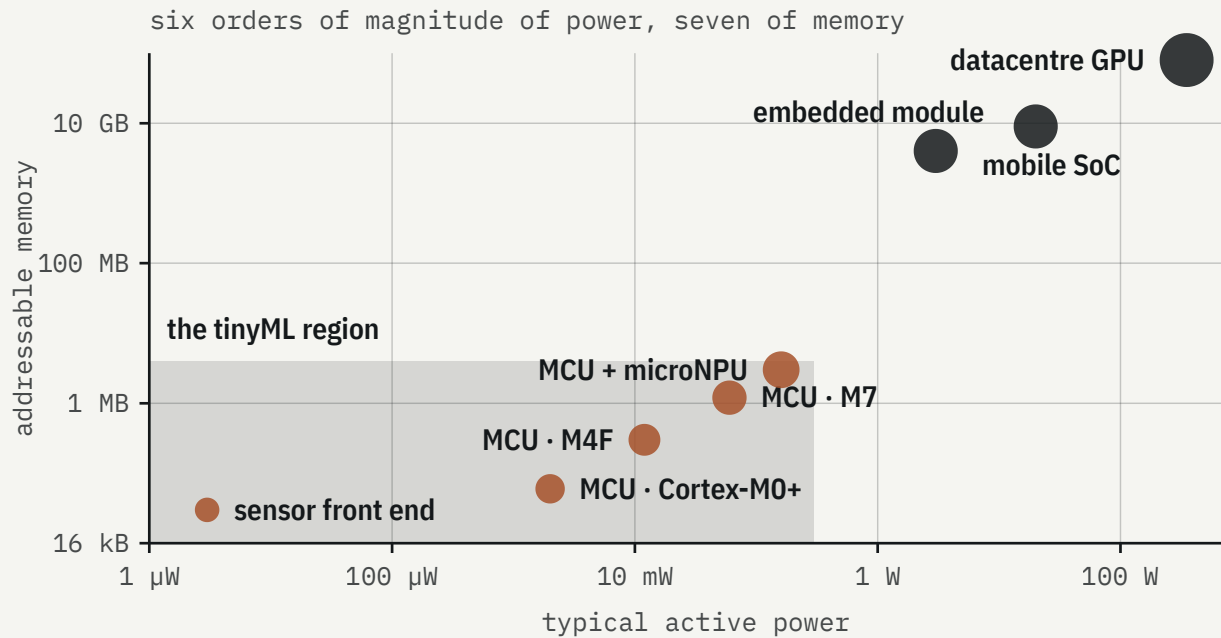
DEFINITION BY CONSTRAINT

Three dimensions that actually matter

- **Timing.** **Hard real-time** fails on a missed deadline; **soft real-time** degrades. Most ML sits soft inside a hard host — and that interface is where the bugs live.
- **Resources.** Kilobytes to a few megabytes. No virtual memory. Often no heap.
- **Energy.** A fixed store and a required service life — which converts directly into a power ceiling.

THE LANDSCAPE

Six orders of magnitude of power, seven of memory



"Small" is never absolute — it is relative to one point on this plane.

"Small" is never absolute. It is relative to **one point on this plane**.

02

What machine learning adds

A primer for those who have not trained models — and a reminder of what matters on a device for those who have.

PRIMER

A model, in one slide

- A **model** is a function $\hat{y} = f(x; \theta)$: input window $\rightarrow$ label, number or score.
- Its form is chosen; its **parameters** θ are learned from labelled examples.
- **Training** searches for θ — expensive, done on a workstation or in the cloud.
- **Inference** evaluates f once for a new input — cheap, done on the device.
- **Generalisation** is the goal: right answers on inputs never seen before.

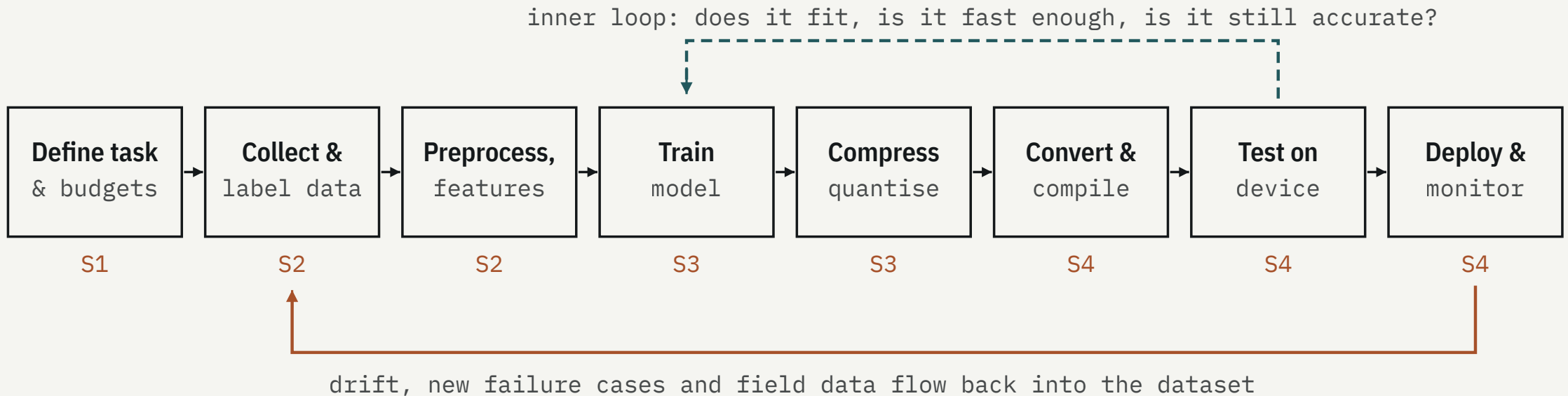
THE DIVISION OF LABOUR

Training happens on a workstation. Inference happens on the device.

When this course says "embedded ML" it almost always means embedded *inference*. Learning on the device is a research frontier (Session 4).

THE WORKFLOW

Most of the effort is in the two loops



Most of the effort in an embedded project sits in the two loops, not in the single "train" box.

S1-S4 mark the session in which each step is treated.

WHY ON THE DEVICE

Five reasons — in the order they are usually the real one

- **Energy of communication:** sending raw data costs more than deciding locally.
- **Privacy and regulation:** data that never leaves cannot leak.
- **Latency and its variance:** control loops care about the tail, not the mean.
- **Availability:** fields, mines, aircraft, oceans.
- **Cost at scale:** bandwidth and cloud inference are recurring costs.

These argue for putting the **decision** on the device — not necessarily the whole model.

03

Applications and workloads

Seven decades of data rate, eight of compute.

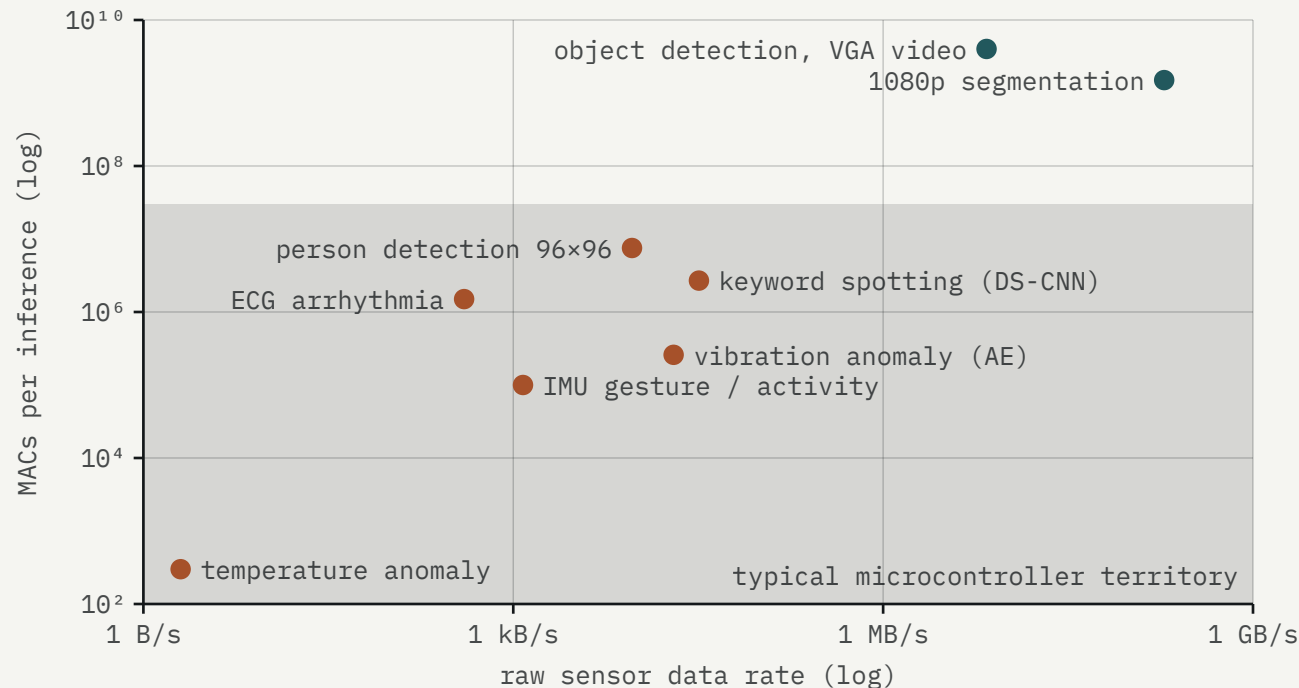
WHERE IT IS DEPLOYED

Application domains and what usually binds

DOMAIN	SENSORS	EXAMPLE TASKS	USUALLY BINDS
Audio and voice	MEMS microphone	keyword spotting, voice activity, acoustic events	always-on energy
Wearables and health	IMU, PPG, ECG	activity, falls, arrhythmia	battery, validation
Industrial monitoring	accelerometer, current	predictive maintenance, anomalies	scarce fault labels
Vision at the edge	low-res camera	person detection, counting	SRAM for activations
Environment, agriculture	gas, humidity, acoustic	air quality, pests	harvested energy
Automotive, robotics	radar, IMU, camera	driver monitoring, gestures	hard deadlines, safety

WORKLOADS

Data rate decides whether streaming is possible; compute decides the device

**Read across:**

data rate decides whether streaming raw data off the device is even possible.

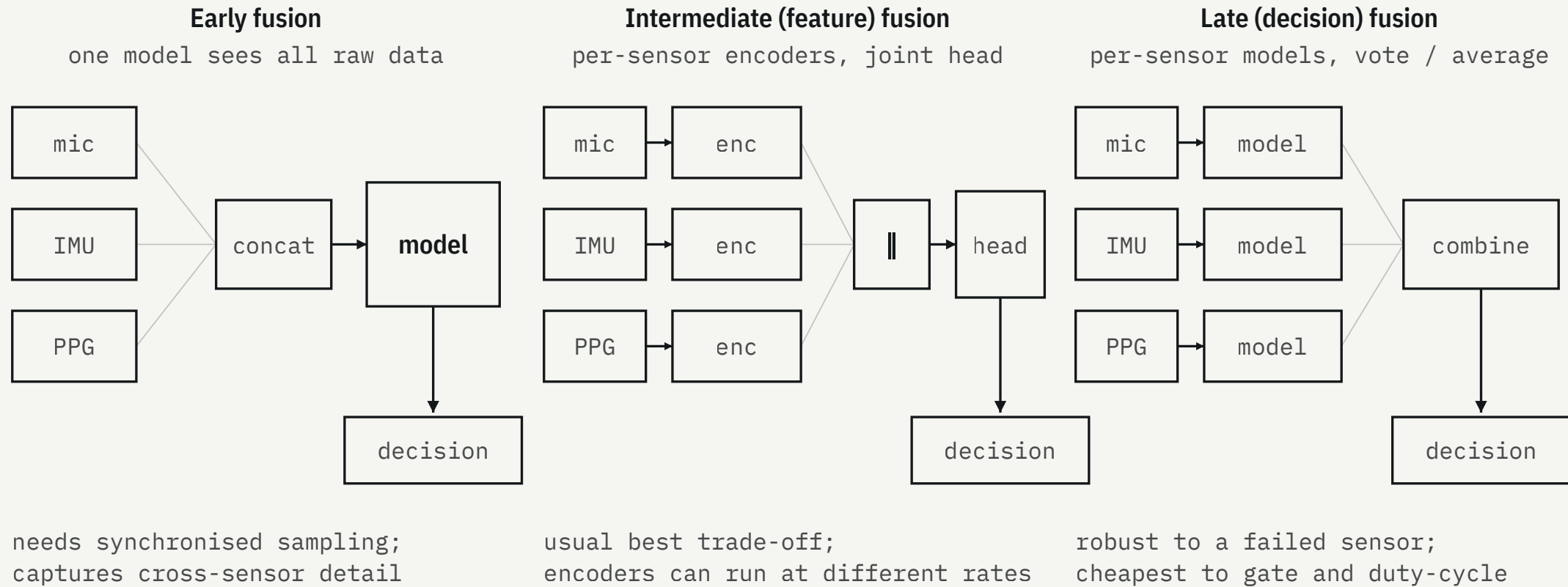
Read up:

compute decides which device class can run the model at all.

Illustrative placement. KWS, person detection and anomaly detection use the MLPerf Tiny reference models; the others are typical published sizes.

SENSOR FUSION

Where to merge several sensors



BREAK

Five minutes.

Next: the five budgets, and where the energy goes.

03

The five budgets

Given a device and an application, write these down before you open a notebook.

BUDGETS

Five numbers, in the order they usually bind

- **Weight memory** — flash. $P \cdot b/8$ bytes plus metadata, interpreter and the rest of the firmware. The model gets about half the flash, never all of it.
- **Peak activation memory** — SRAM. **Not the model size.** This is the one that kills naive designs.
- **Compute** — MACs, divided by the device's **sustained** rate, never its peak. A lower bound only.
- **Latency** — set by the application, and it must include feature extraction.
- **Energy per inference** — and the duty cycle that follows from it.

THE BUDGET PEOPLE GET WRONG

Peak activation memory

$$\text{peak_SRAM} = \max \text{ over layers } i \text{ of } (|x_i| + |y_i| + \sum |t| \text{ for } t \text{ still live })$$
$$\text{Conv2D MACs} = H \cdot W \cdot k^2 \cdot C_{\text{in}} \cdot C_{\text{out}}$$

Parameters live at the back of a CNN; activations peak at the front, where the spatial resolution is still high. That asymmetry is the single most important structural fact about putting CNNs on microcontrollers.

SAY IT ONCE, CLEARLY

Peak activation memory, not parameter count, decides whether a CNN fits on a microcontroller.

Parameters at the back, activations at the front. A model can be tiny and still not run.

WORKED DERIVATION

A one-year coin cell

1. A CR2032 is nominally 3 V with about 225 mAh: **$E = 0.225 \times 3 \times 3600 \approx 2.4 \times 10^3 \text{ J}$**
2. One year is $3.15 \times 10^7 \text{ s}$, so the average power ceiling is **$P \leq 2.4 \times 10^3 / 3.15 \times 10^7 \approx 77 \text{ } \mu\text{W}$**
3. Standby (RTC, sensor bias, leakage) already takes $20 \text{ } \mu\text{W}$ → **$57 \text{ } \mu\text{W}$ left for inference**
4. If one inference costs 5 mJ: **$f \leq 57 \text{ } \mu\text{W} / 5 \text{ mJ} \approx 0.011 \text{ Hz}$** — one inference every ~90 s

The application wants 1 Hz. You are short by a factor of about 90.

THE FACTOR OF 90

Four legitimate ways out — and one that is not

- **Reduce energy per inference** — compression and efficient architectures. Sessions 8–12.
- **Reduce the rate** — cascade with a cheap always-on stage. §1.5, and in ten minutes.
- **Increase the store** — a bigger cell, or energy harvesting.
- **Relax the specification** — the most underused option in the industry.
- **Not on the list:** “train a better model”.

04

Where the energy actually goes

Students arrive believing arithmetic is expensive. It is not, and the gap is two orders of magnitude.

FOUNDATIONS · HOROWITZ, ISSCC 2014 · 45 NM

Energy per operation

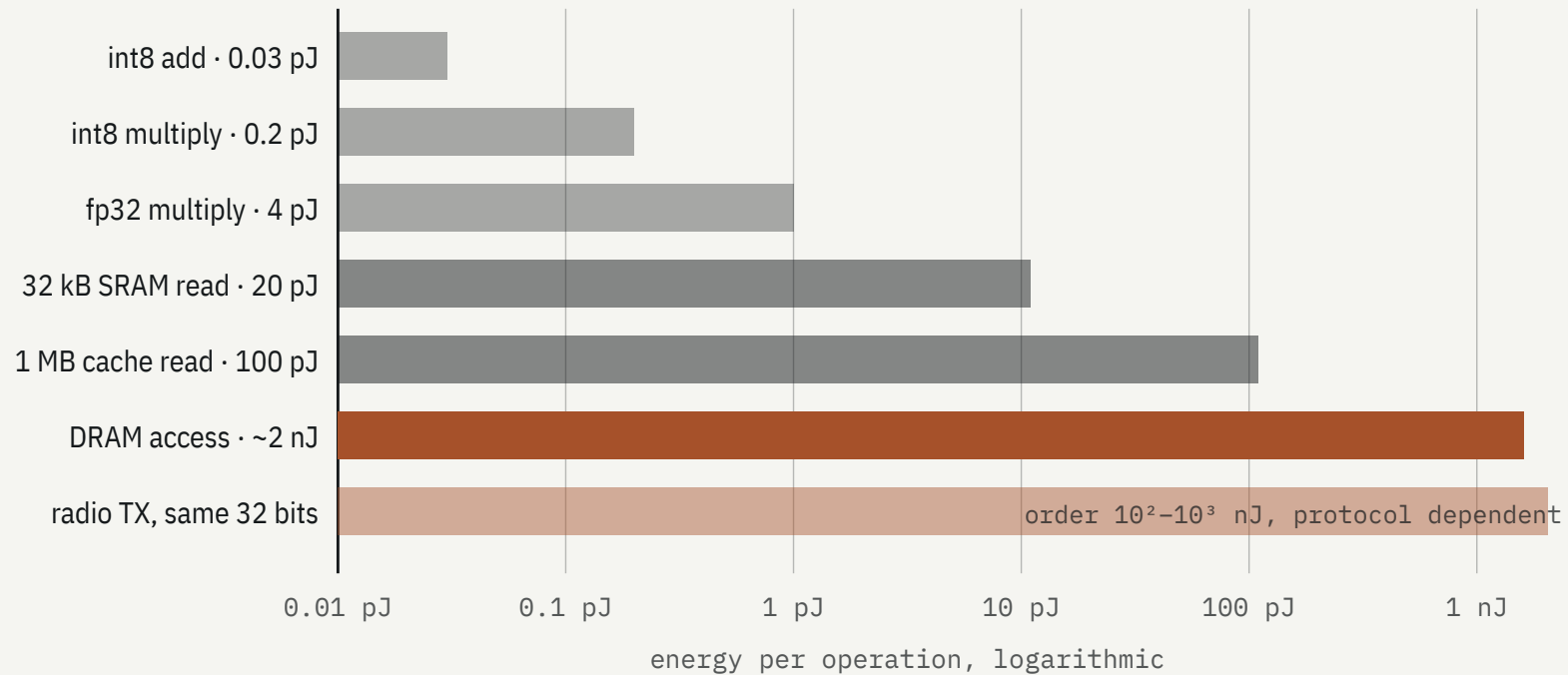
OPERATION	ENERGY	OPERATION	ENERGY
int8 add	0.03 pJ	32 kB cache read	20 pJ
int32 add	0.1 pJ	1 MB cache read	100 pJ
int8 multiply	0.2 pJ	8 kB cache read	10 pJ
fp16 multiply	1 pJ	DRAM access	1.3–2.6 nJ
fp32 multiply	4 pJ	<i>ratio, DRAM : int8 add</i>	<i>≈ 50 000×</i>

Absolute values shrink with process node. The **ratios** have proved remarkably stable.

THE COST HIERARCHY

Arithmetic on the left. Moving the same operands on the right.

45 nm reference values, after Horowitz (ISSCC 2014)



Radio is worse than DRAM. This bar chart is the quantitative case for edge inference.

THREE CONCLUSIONS

What follows from that picture

- **Precision is an energy knob.** fp32 → int8 multiply is roughly 20× in the multiplier alone, before the halved memory traffic. **Session 3.**
- **Locality is a bigger knob than precision.** Any technique that keeps operands on-chip beats any technique that merely reduces arithmetic. **Sessions 6–7.**
- **Sparsity only pays if the hardware can skip.** Removing a multiply saves 0.2 pJ; the index bookkeeping may cost more. **Session 3.**

05

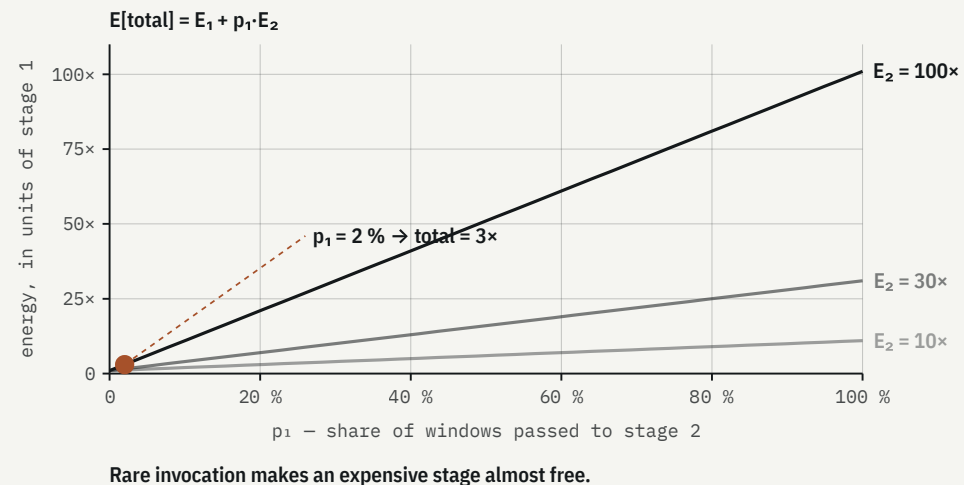
The cascade

The architectural pattern that falls straight out of the energy picture.

EXPECTED ENERGY

Why a 100× more expensive stage can be nearly free

- $E[\text{total}] = E_1 + p_1 \cdot E_2 + p_1 p_2 \cdot E_3 + \dots$
- Most windows are negative: no keyword, no person, no fault.
- With $p_1 = 2\%$, a stage 100× more expensive costs only **2× the cheap stage**.
- So the always-on stage is tuned for **recall at fixed energy**. Precision is the next stage's job.



04

Three systems, end to end

The budgets only become real when applied.

CASE STUDIES

Three systems, three different binding constraints

Keyword spotting on a coin cell

- 16 kHz mic → 49×10 MFCC map
- DS-CNN: ≈39 k params, 2.7 M MACs
- Cortex-M4F class MCU
- **Binds: energy** — fixed by a voice-activity cascade

Vibration anomaly on a motor

- 3-axis accelerometer, 1–10 kHz
- band energies per second
- autoencoder trained on healthy data
- **Binds: data** — no labelled failures, drifting baseline

Arrhythmia in a wearable

- single-lead ECG, 250 Hz
- intervals + tree, or a 1-D CNN
- wearable SoC
- **Binds: validation** — sensitivity on clinical data

EXERCISE · 4 MINUTES, IN PAIRS

Find the binding budget

1. Pick one of the three systems.
2. Write the five budgets with an order of magnitude each: flash, peak SRAM, MACs, latency, energy per inference.
3. Circle the one that binds.
4. Change one thing — a camera instead of a microphone, mains power instead of a coin cell, 10× more data — and say which budget binds now.

There is rarely one right answer; there is always a wrong one: not writing them down.

CONTEXT

Four inflection points, honestly told

- **2016–17** — practical mobile-scale CNNs. **Session 3.**
- **2017–18** — integer-only inference and a toolchain that made it routine. **Session 3.**
- **2020 →** — memory-aware co-design of architecture **and** runtime for MCUs. **Session 4.**
- **2023 →** — transformers, state-space models and language models at the edge. Open research, not settled practice. **Session 4.**

Signal processing on microcontrollers is decades old. What changed is which functions we are willing to **learn** rather than design.

WHERE A PHD FITS

Open problems, by session

- **On-device learning** within kB of memory, without labels (S4).
- **Data efficiency:** self-supervision, few-shot adaptation, synthetic data (S2–S3).
- **Hardware-aware design:** NAS and compression on measured latency and energy (S3).
- **Compilers and runtimes** for heterogeneous MCU + NPU systems (S4).
- **Beyond digital:** analog in-memory and neuromorphic hardware (S4).
- **Trust:** robustness, privacy, secure updates — and honest **measurement**.

BEFORE SESSION 2

Reading, and choosing a paper

V. Sze, Y.-H. Chen, T.-J. Yang, J. Emer, “Efficient Processing of Deep Neural Networks: A Tutorial and Survey,” *Proceedings of the IEEE* 105(12), 2017 — §I–III.
Optional: P. Warden, “Speech Commands,” 2018.

Why: The canonical statement that efficiency is a hardware–algorithm co-design problem, and the vocabulary — dataflow, reuse, energy per access — of Session 4.

Shortlist two presentation papers from the website; send your choice by the end of Session 2.

IF YOU REMEMBER ONE THING

Write the five budgets down before you write any code. Arithmetic is nearly free; data movement and communication are not. When the numbers do not close, the four legitimate moves are cheaper inference, fewer inferences, more energy, or a different specification — never “train a better model”.

BEFORE YOU GO

Think about these

- Q1 The “privacy” argument for edge inference is often made loosely. Construct a case where moving inference to the device makes privacy **worse**.
- Q2 A vendor advertises “2 TOPS at 1 W”. Name four things you must know before that number predicts anything about your application.
- Q3 For your own research problem: which fusion strategy, and which budget binds first?